The Journal of Open Source Software

Back to sequences: find the origin of k-mers

Anthony Baire[1], **Pierre Marijon**[2], **Francesco Andreace**[3], **and Pierre Peterlongo**[1]

1 Univ. Rennes, Inria, CNRS, IRISA - UMR 6074, Rennes, F-35000 France **2** Laboratoire de Biologie Médicale Multisites SeqOIA, Paris, France **3** Department of Computational Biology, Institut Pasteur, Université Paris Cité, Paris, F-75015, France

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 10 June 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Abstract

A vast majority of bioinformatics tools dedicated to the treatment of raw sequencing data heavily use the concept of k-mers, which are words of length k. This enables us to reduce the redundancy of data (and thus the memory pressure), to discard sequencing errors, and to dispose of objects of fixed size that can be easily manipulated and compared to each other. A drawback is that the link between each k-mer and the original set of sequences to which it belongs is lost. Given the volume of data considered in this context, finding back this association is costly. In this work, we present "back_to_sequences", a simple tool designed to index a set of k-mers of interest and to stream a set of sequences, extracting those containing at least one of the indexed k-mer. In addition, the occurrences of k-mers in the sequences can be provided. Our results show that back_to_sequences streams ≈ 200 short read per millisecond, allowing to search k-mers in hundreds of millions of reads in a matter of a few minutes.

Availability: github.com/pierrepeterlongo/back_to_sequences

Statement of need

In the 2010s, following the emergence of next-generation sequencing technology, read assembly strategies based on the overlap-layout-consensus paradigm (OLC) were unable to scale to tens of millions of reads or more, prompting the usage of the *de Bruijn* graph (dBG) data structure (Flicek & Birney, 2009; Schatz et al., 2010). The success of dBG was due to the fact that the main difficulties associated with the nature of the sequencing data (read redundancy, nonuniform coverage and nonuniform overlap between reads, sequencing errors, unknown sequencing strand) were complex to handle with OLC while being easy to handle or simply solved with the dBG approach (Li et al., 2012).

Recall that in the dBG assembly approach, 1. all k-mers(words of length k) from a set of reads are counted; 2. those with an abundance lower than a threshold are considered to contain sequencing errors and are discarded; 3. the remaining k-mers are organized in a dBG; 4. the paths of the dBG form the basis of the assembly, later improved thanks to scaffolding tools (Huson et al., 2002) such as the tools provided, for instance, by the Spades assembler (Bankevich et al., 2012).

The usefulness of k-mers did not end with their use in dBGs. A large and redundant set of sequences, such as a sequencing read set, can be summarized by its set of k-mers. Among multiple fundamental tasks, it has been the basis for metagenome comparisons (Benoit et al., 2016), for taxonomy characterization (Wood et al., 2019), for indexing purposes (Cracco & Tomescu, 2023; Lemane, Medvedev, et al., 2022), for genotyping (Grytten et al., 2022), for species identification (Sarmashghi et al., 2019), for transcript expression estimation (Zhang &

Wang, 2014), or for variant discovery (Uricaru et al., 2015) to cite only a few.

One of the keys to the success of the use of k-mers is its low resource needs. Whatever the sequencing coverage, once filtered, the number of distinct k-mers is at most equal to the original genome size. This offers a minimal impact on RAM and/or disk needs. However, this comes at the cost of losing the link between each k-mer and the sequences(s) from which it originates. Storing explicitly these links would reintroduce the problem associated with the abundance of original reads, as the link between each original read and each of its k-mers would have to be stored. For instance, considering k-mers from a sequencing experiment of a human genome (≈ 3 billions nucleotides) with a coverage of 50x (each k-mer occurs on average in 50 distinct reads) would require more than 2Tb of space considering 64 bits for storing each link and 64 bits for storing the associated read identifier. This is not acceptable.

There are many situations in which finding the origin of k-mers from a set in a set of reads is informative. For instance, this is the case in studies in which the output is composed of k-mers associated to biological knowledge such as biological variants (Uricaru et al., 2015), or k-mers specific to a phenotypic trait (Lemane, Chikhi, et al., 2022) to cite a few. This approach can also be used for quality control (Plaza Onate et al., 2015) or contamination removal (González et al., 2023) for instance.

Finding back the link between a k-mer and each original read in which it occurs can be performed by indexing the reads (Marchet et al., 2020) which hardly scales hundreds of millions reads. One may also apply *grep-like* evolved pattern matching approaches such as `pt` (Monochromegane, 2018). However, even if they were highly optimized in recent decades, these approaches cannot efficiently detect thousands of k-mers in millions of reads. The approaches that use k-mers for genotyping such as `kage` (Grytten et al., 2022) may find the number of occurrences of k-mers but not extract the sequences from which they originate.

In this context, we propose `back_to_sequences`, a tool specifically dedicated to extracting from $\mathcal{S}$, a set of sequences (eg. reads), those that contain some of the k-mers from a set $\mathcal{K}$ given as input. The occurrences of k-mers in each sequence of the queried set $\mathcal{S}$ can also be output. In addition to this fundamental task, additional features are described in the following.

Results

The tool we introduce, `back_to_sequences`, uses the native `rust` data structures to index and query k-mers. Its advantages are 1. its simplicity (install and run); 2. its low RAM usage; 3. its fast running time; 4. its additional features adapted to k-mers: control of their orientation (see below) and sequence filtration based on the minimal and maximal percent of k-mers shared with the indexed set. To the best of our knowledge, there exists no other tool dedicated to this specific task.

In the general case, the DNA sequence orientation is unknown. DNA can be sequenced either in one strand (eg $AAGGC$) or in the reverse complement strand, read from right to left and commutating A with T, and C with G (eg $GCCTT$). This is why k-mers from $\mathcal{S}$ and $\mathcal{K}$ can be considered either as original or as *canonical*, the smallest alphabetical word between the k-mer and its reverse complement.

We propose a simple performance benchmark to assess the `back_to_sequences` scaling performances on random sequences and on real metagenomic data.

Benchmark on random sequences

We generated a random sequence s on the alphabet $\{A, C, G, T\}$ of size 100 million base pairs. From this sequence s, we randomly extracted 50,000 sub-sequences each of size 50. We consider these sequences as containing the set $\mathcal{K}$ of k-mers to be searched. As we used

$k = 31$, each subsequence contains $50 - 31 + 1 = 20 kmers$. Doing so, we consider a set of at most $50000 \times 20 = 1,000,000$ k-mers.

We also generated six sets: $\{\mathcal{S}_{10k}, \mathcal{S}_{100k}, \mathcal{S}_{1M}, \mathcal{S}_{10M}, \mathcal{S}_{100M}, \mathcal{S}_{200M}\}$, composed respectively of 10 thousand, 100 thousand, one million, 10 million, 100 million, and 200 million sequences, each of length 100 nucleotides. Each sequence is randomly sampled from s.

In each sequence of each set $\mathcal{S}_i$, we searched for the existence of k-mers indexed in $\mathcal{K}$. The performances are provided Table 1. Presented results were obtained on the GenOuest platform on a node with 32 threads Xeon 2.2 GHz, on a MacBook, Apple M2 pro, 16 GB RAM with 10 threads, and an AMD Ryzen 7 4.2 GHz 5800X 64 GB RAM with 16 threads, respectively denoted by "Time GenOuest", "Time mac" and "Time AMD" in this table. These results highlight the scalability of the `back_to_sequences` tool, able to search a million of k-mers in hundreds of millions of reads on a laptop computer in a matter of dozens of minutes with negligible RAM usage.

Table 1: The `back_to_sequences` performances searching one millions k-mers in 10 thousand to 100 million reads. Tested version: 2.6.0.

Number of reads	Time GenOuest	Time mac	Time AMD	max RAM
10,000	0.7s	0.5s	0.4s	0.13 GB
100,000	0.8s	0.8s	1.2s	0.13 GB
1,000,000	2.0s	3.5s	7,1s	0.13 GB
10,000,000	7.1s	11s	16s	0.13 GB
100,000,000	47s	58s	48s	0.13 GB
200,000,000	1m32	1m52	1m44	0.13 GB

Benchmark on *Tara* ocean seawater metagenomic data

The previously proposed benchmark shows the scalability of the proposed approach. Although performed on random sequences, there is no objective reason why performance should differ on real data, regardless of the number of k-mers actually detected in the data. We verify this claim by applying `back_to_sequences` to real complex metagenomic sequencing data from the *Tara* ocean project (Sunagawa et al., 2020).

We downloaded one of the *Tara* ocean read sets: station number 11 corresponding to a surface Mediterranean sample, downloaded from the European Nucleotide Archive, identifier ERS488262[1]. We extracted the first 100 million reads, which are all of length 100. Doing so, this test is comparable to the result presented Table 1 querying 100 million random reads. Using `back_to_sequences` we searched in these reads each of the 69 31-mers contained in its first read. On the GenOuest node, `back_to_sequences` enabled to retrieve all reads that contain at least one of the indexed k-mers in 5m17 with negligible RAM usage of 45MB. As expected, these scaling results are in line with the results presented Table 1.

Out of curiosity, we ran `back_to_sequences` on the full read set, composed of ≈ 26.3 billion k-mers, and 381 million reads, again for searching the 69 k-mers contained in its first read. This operation took 20m11.

Possible alternatives

To the best of our knowledge, there exists no tool specifically dedicated to this task, while indexing the set of k-mers$\mathcal{K}$.

Genotypers as kage (Grytten et al., 2022), using `kmer mapper` (Ivar Grytten, 2020), provide a way to count the number of occurrences of each k-mer from a set of reference k-mers in a read file. However, they do not offer the feature to extract reads that contain any reference k-mer.

[1] http://www.ebi.ac.uk/ena/data/view/ERS488262

Finding one unique k-mer of interest in a set of sequences can be done using the classical `grep` or more recent pattern-matching tools such as "*The Platinum Searcher*" (Monochromegane, 2018) or "*The Silver Searcher*" (Greer, 2020).

As for testing, on the MacBook, Apple M2 pro, we queried one k-mer in the $\mathcal{S}_{100M}$ dataset (see Section 2.1) using `grep`, *The Platinum Searcher*, and *The Silver Searcher*.

- `grep` required 44 seconds. Thus, by simple extrapolation, searching for one million k-mers on a single computer would require approximately 500 days, to be compared to 5 to 6 minutes using `back_to_sequences`.
- `pt` (*The Platinum Searcher*) required 15 seconds, which can be extrapolated to approximately 175 days if searching for one million k-mers.
- `ag` (*The Silver Searcher*) did not finish after 400 seconds.

Summing up, these alternative tools are not meant for querying numerous patterns at the same time and do not scale to large problem instances.

Note also that these alternative tools are not specialized for genomic data in which one is interested in searching for a k-mer and potentially its reverse complement. Finally, these tools do not easily provide the number of occurrences or occurrence positions of each of the searched patterns when there are many.

Method and features

`back_to_sequences` is written in `rust`. It uses the native HashMap for storing the searched k-mer set, with alternative aHash (Zhao et al., 2019) hash function. Depending on the user choice, the original or the canonical version of each k-mer from the "reference-k-mers" set is indexed. The source code is unit-tested and functionally tested using tools from the Rust community.

At query time, given a sequence s from $\mathcal{S}$, all its k-mers are extracted and queried. Depending on the user's choice, the canonical or the original representation of each k-mer from the reference set is indexed. Again, depending on the user's choice, for each queried k-mer, the original version, its reverse complement, or its canonical representation is queried. If the queried k-mer belongs to the index, its number of occurrences is increased. The number of k-mers extracted from the sequence s that have a match with the index is output together with the original sequence. As an option, in addition to only counting the number of occurrences of each k-mer from $\mathcal{K}$ in $\mathcal{S}$, `back_to_sequences` enables to also record their occurrence positions and orientation and to output this information. Sequences are queried in parallel.

A minimal and a maximal threshold can be fixed by the user. A queried sequence is output only if its percentage of k-mers that belong to the searched k-mers is strictly higher than the minimal threshold and lower or equal to the maximal threshold. While the minimal threshold enables to focus on sequences that are similar enough with a set of k-mers, the maximal threshold offers for instance a way to remove contaminated sequences.

The sequences to be queried can be provided as a fasta or fastq file (gzipped or not). They can also be read directly from the standard input (*stdin*). This offers the may to stream sequences as they arrive, for instance when they are obtained during an Oxford Nanopore sequencing process.

The output format of queried sequences respects the input format: if the input is in fasta (resp. fastq), the output is in fasta (resp. fastq).

Conclusion

We believe that back_to_sequences is a generic and handy tool that will be beneficial for building pipelines that require manipulating k-mers and finding back the sequences from which they originate and/or counting their number of occurrences in a set of genomic sequences. We also believe that back_to_sequences will have other straightforward applications, such as quality control, contamination removal, or genotyping known pieces of sequences in raw sequencing datasets, all of which being possible in real time throughout the sequencing process.

Acknowledgements

We acknowledge the GenOuest core facility (https://www.genouest.org) for providing the computing infrastructure. The work was funded by ANR SeqDigger (ANR-19-CE45-0008), and by the Inria Challenge "OmicFinder" (https://project.inria.fr/omicfinder/).

Bankevich, A., Nurk, S., Antipov, D., Gurevich, A. A., Dvorkin, M., Kulikov, A. S., Lesin, V. M., Nikolenko, S. I., Pham, S., Prjibelski, A. D., & others. (2012). SPAdes: A new genome assembly algorithm and its applications to single-cell sequencing. *Journal of Computational Biology*, *19*(5), 455–477. https://doi.org/10.1089/cmb.2012.0021

Benoit, G., Peterlongo, P., Mariadassou, M., Drezen, E., Schbath, S., Lavenier, D., & Lemaitre, C. (2016). Multiple comparative metagenomics using multiset k-mer counting. *PeerJ Computer Science*, *2*, e94. https://doi.org/10.7717/peerj-cs.94

Cracco, A., & Tomescu, A. I. (2023). Extremely fast construction and querying of compacted and colored de bruijn graphs with GGCAT. *Genome Research*, gr–277615. https://doi.org/10.1101/gr.277615.122

Flicek, P., & Birney, E. (2009). Sense from sequence reads: Methods for alignment and assembly. *Nature Methods*, *6*(Suppl 11), S6–S12. https://doi.org/10.1038/nmeth.1376

González, C. D., Rangavittal, S., Vicedomini, R., Chikhi, R., & Richard, H. (2023). aKmerBroom: Ancient oral DNA decontamination using bloom filters on k-mer sets. *Iscience*, *26*(11). https://doi.org/10.1016/j.isci.2023.108057

Greer, G. (2020). *The Silver Searcher*. https://github.com/ggreer/the_silver_searcher.

Grytten, I., Dagestad Rand, K., & Sandve, G. K. (2022). KAGE: Fast alignment-free graph-based genotyping of SNPs and short indels. *Genome Biology*, *23*(1), 209. https://doi.org/10.1186/s13059-022-02771-2

Huson, D. H., Reinert, K., & Myers, E. W. (2002). The greedy path-merging algorithm for contig scaffolding. *Journal of the ACM (JACM)*, *49*(5), 603–615. https://doi.org/10.1145/585265.585267

Ivar Grytten, K. D. R. (2020). *Kmer Mapper*. https://github.com/ivargr/kmer_mapper.

Lemane, T., Chikhi, R., & Peterlongo, P. (2022). Kmdiff, large-scale and user-friendly differential k-mer analyses. *Bioinformatics*, *38*(24), 5443–5445. https://doi.org/10.1093/bioinformatics/btac689

Lemane, T., Medvedev, P., Chikhi, R., & Peterlongo, P. (2022). Kmtricks: Efficient and flexible construction of bloom filters for large sequencing data collections. *Bioinformatics Advances*, *2*(1), vbac029. https://doi.org/10.1093/bioadv/vbac029

Li, Z., Chen, Y., Mu, D., Yuan, J., Shi, Y., Zhang, H., Gan, J., Li, N., Hu, X., Liu, B., & others. (2012). Comparison of the two major classes of assembly algorithms: Overlap–layout–consensus and de-bruijn-graph. *Briefings in Functional Genomics*, *11*(1), 25–37. https://doi.org/10.1093/bfgp/elr035

DRAFT

Marchet, C., Lecompte, L., Limasset, A., Bittner, L., & Peterlongo, P. (2020). A resource-frugal probabilistic dictionary and applications in bioinformatics. *Discrete Applied Mathematics*, *274*, 92–102. https://doi.org/10.1016/j.dam.2018.03.035

Monochromegane. (2018). *The Platinum Searcher*. https://github.com/monochromegane/the_platinum_searcher.

Plaza Onate, F., Batto, J.-M., Juste, C., Fadlallah, J., Fougeroux, C., Gouas, D., Pons, N., Kennedy, S., Levenez, F., Dore, J., & others. (2015). Quality control of microbiota metagenomics by k-mer analysis. *BMC Genomics*, *16*, 1–10. https://doi.org/10.1186/s12864-015-1406-7

Sarmashghi, S., Bohmann, K., P. Gilbert, M. T., Bafna, V., & Mirarab, S. (2019). Skmer: Assembly-free and alignment-free sample identification using genome skims. *Genome Biology*, *20*, 1–20. https://doi.org/10.1186/s13059-019-1632-4

Schatz, M. C., Delcher, A. L., & Salzberg, S. L. (2010). Assembly of large genomes using second-generation sequencing. *Genome Research*, *20*(9), 1165–1173. https://doi.org/10.1101/gr.101360.109

Sunagawa, S., Acinas, S. G., Bork, P., Bowler, C., Coordinators, T. O., Eveillard, D., Gorsky, G., Guidi, L., Iudicone, D., Karsenti, E., Lombard, F., Ogata, H., Pesant, S., Sullivan, M. B., Wincker, P., & Vargas, C. de. (2020). Tara oceans: Towards global ocean ecosystems biology. *Nat Rev Microbiol*, *18*(8), 428–445. https://doi.org/10.1038/s41579-020-0364-5

Uricaru, R., Rizk, G., Lacroix, V., Quillery, E., Plantard, O., Chikhi, R., Lemaitre, C., & Peterlongo, P. (2015). Reference-free detection of isolated SNPs. *Nucleic Acids Research*, *43*(2), e11–e11. https://doi.org/10.1093/nar/gku1187

Wood, D. E., Lu, J., & Langmead, B. (2019). Improved metagenomic analysis with kraken 2. *Genome Biology*, *20*, 1–13. https://doi.org/10.1186/s13059-019-1891-0

Zhang, Z., & Wang, W. (2014). RNA-skim: A rapid method for RNA-seq quantification at transcript level. *Bioinformatics*, *30*(12), i283–i292. https://doi.org/10.1093/bioinformatics/btu288

Zhao, C., Gui, J., Guo, Y., Jiang, J., Yang, T., Cui, B., & Zhang, G. (2019). *AHash: A load-balanced one permutation hash*. https://openreview.net/forum?id=rJe9fTNtPS

DRAFT